

Guía teórica para principiantes: captura estereoscópica de foto y vídeo en Meta Quest (Unity)

Documento de estudio complementario al [PLAN_IMPLEMENTACION.md](#) Objetivo: explicar, desde cero y con detalle, todos los conceptos teóricos que aparecen en el plan (objetivos y soluciones), de modo que una persona sin experiencia pueda entenderlos y llegar a dominarlos. Cada concepto nuevo se acompaña de una referencia en línea que lo respalda.

Cómo leer esta guía: puedes ir en orden (está pensada como un curso) o saltar a la sección que te interese. Las **preguntas concretas del enunciado** —cómo se graba y almacena el vídeo, qué significa "descartar al saturarse la cola", si se afecta la memoria de vídeo o la de disco, qué es "fabricar FPS 1/intervalo", "blit", "emparejamiento", "B08/B09"— se responden sobre todo en las secciones **4, 5, 6 y 7**.

Índice

0. El proyecto en una frase
 1. Estereoscopía: cómo dos imágenes producen profundidad
 2. La plataforma: Quest, Horizon OS, Unity, Meta XR, MRUK, Passthrough, URP y OpenXR
 3. Imágenes en la GPU: texturas, materiales, shaders y qué es un *blit*
 4. De la GPU a la CPU: *readback*, hilos y el porqué de B08/B09
 5. Emparejamiento estéreo: sincronizar el ojo izquierdo y el derecho
 6. Memoria y tiempo: la sección clave
 7. Codificación de vídeo: códec, contenedor, MediaCodec y YUV
 8. Foto: PNG frente a JPEG y los metadatos
 9. Almacenamiento y publicación: del archivo temporal a la galería
 10. Arquitectura, estados y asincronía: cómo se coordina todo
 11. La interfaz en realidad virtual
 12. Cómo leer el plan: prioridades, identificadores y criterios
 13. Glosario rápido
 14. Referencias consolidadas
-

0. El proyecto en una frase

Las Meta Quest 3 y 3S tienen **dos cámaras frontales** (una alineada con cada ojo). Este Trabajo de Fin de Grado construye una aplicación de Unity que toma esas dos imágenes, las junta y las guarda como **fotos y vídeos con sensación de profundidad** (3D), y ofrece un **visor propio** que envía a cada ojo su imagen correspondiente.

Todo lo demás —texturas, *blit*, *readback*, colas, códecs, timestamps— son las piezas técnicas necesarias para lograr eso de forma correcta, fluida y sin agotar la memoria del dispositivo.

1. Estereoscopia: cómo dos imágenes producen profundidad

La idea básica

Tienes dos ojos separados unos **6,3 cm** de media (esa distancia se llama **distancia interpupilar** o *IPD*). Cada ojo ve el mundo desde un punto ligeramente distinto, así que las dos imágenes no son idénticas: un objeto cercano se "mueve" mucho entre una vista y otra, y uno lejano casi nada. Esa diferencia de posición entre las dos vistas se llama **disparidad binocular**. Tu cerebro mide esa disparidad y de ahí deduce **a qué distancia está cada cosa**. Eso es la visión estereoscópica. [1][2][3]

La **estereoscopia** artificial imita ese truco: si capturas dos fotos desde dos puntos separados (dos cámaras) y luego consigues que **el ojo izquierdo vea solo la foto izquierda y el derecho solo la derecha**, el cerebro reconstruye la profundidad como si estuvieras allí. [1]

SBS (*Side-by-Side*, "lado a lado")

La forma más simple de guardar un par estéreo en **un solo archivo** es poner las dos vistas **una al lado de la otra**: la mitad izquierda de la imagen es lo que verá el ojo izquierdo y la mitad derecha lo que verá el derecho. Eso es **SBS**. En el plan aparece constantemente ("PNG SBS", "MP4 SBS"): significa exactamente eso, dos vistas concatenadas horizontalmente. [1]

- Ventaja: un único archivo estándar (un PNG, un MP4) contiene las dos vistas.
- Consecuencia: la imagen SBS es **el doble de ancha** que una vista sola. Si cada ojo es 1280×960, el SBS es 2560×960. Por eso en el código verás multiplicaciones por 2: `AnchoOjo * 2`.

Para **reproducir** un SBS en el visor, el shader hace lo contrario: al ojo izquierdo le muestra la franja de coordenadas horizontales `[0, 0.5]` y al derecho `[0.5, 1]` (ver §3 y §11).

Por qué se usan las cámaras físicas (y no las virtuales)

Un error inicial del proyecto (bloqueo **B05** en el plan) era generar el par estéreo renderizando cámaras *virtuales* de Unity. Eso capturaría el mundo virtual, no **el entorno real** que el usuario ve por *passthrough*. El objetivo O1 exige capturar la realidad, así que hay que leer las **dos cámaras físicas** del visor (§2).

Referencias: Estereoscopia [1]; Disparidad binocular [2]; Distancia interpupilar [3].

2. La plataforma

Antes de entrar en píxeles conviene situar el "ecosistema" de nombres que aparece en el plan.

Nombre	Qué es	Papel en el proyecto
Meta Quest 3 / 3S	Gafas de realidad mixta autónomas (llevan dentro un ordenador Android con procesador Snapdragon).	El hardware objetivo. Tienen las dos cámaras que se van a capturar.
Horizon OS	El sistema operativo de las Quest (una variante de Android).	Concede o deniega el permiso de cámara (<code>horizonos.permission.HEADSET_CAMERA</code>). [7]
Unity 6	Motor de videojuegos/3D con el que está hecha la app.	Marco donde vive todo el código C#, las escenas y los materiales.
Meta XR SDK / MRUK 83	Bibliotecas de Meta para Unity. MRUK = <i>Mixed Reality Utility Kit</i> .	Da acceso a las cámaras físicas mediante la Passthrough Camera API . [4][5]
Passthrough	La función que muestra el vídeo de las cámaras exteriores dentro del visor (ver "a través" de las gafas).	La fuente de imagen real que se captura. [6]
URP (<i>Universal Render Pipeline</i>)	La "tubería" de render de Unity que decide cómo se dibuja cada fotograma.	Define el espacio de color y cómo se comportan shaders y <i>blits</i> . [8]
OpenXR	Estándar abierto para VR/AR (una "lengua común" entre apps y visores).	Capa sobre la que se apoyan Unity y Meta para hablar con el hardware. [9]

Passthrough Camera API (PCA) y el permiso

La **Passthrough Camera API** es lo que permite a la app **leer las imágenes de las cámaras**, no solo verlas. Es relativamente nueva y **exige permiso explícito** del usuario, igual que cualquier app de móvil que quiere usar la cámara. El plan insiste (B12, F-varios) en pedir el permiso de verdad, esperar la respuesta real y reflejar en pantalla si se deniega. En el código, `GestorCaptura` hace exactamente eso con `Permission.RequestUserPermission("horizonos.permission.HEADSET_CAMERA", ...)`. [4][7]

La API local (MRUK 83) expone, por cada cámara, propiedades como `IsSupported`, `IsPlaying`, `CurrentResolution`, `Timestamp`, `IsUpdatedThisFrame` y un método `GetTexture()` que entrega la imagen **como una textura de GPU**. Un detalle importante que documenta el propio SDK: **la textura se actualiza en el hilo de render**, así que si la lees en el momento equivocado puedes recoger el fotograma anterior (esto reaparece en §4). [4]

Referencias: Passthrough Camera API [4]; MRUK [5]; Passthrough [6]; permisos Android [7]; URP [8]; OpenXR [9].

3. Imágenes en la GPU: texturas, materiales, shaders y qué es un *blit*

Esta sección desmonta la palabra **"blit"** y las que la rodean.

3.1 ¿Dónde viven las imágenes? CPU, GPU y VRAM

Un ordenador tiene dos "cerebros" que trabajan con la imagen:

- La **CPU** (procesador general): ejecuta la lógica del programa (C#). Trabaja con la memoria normal, la **RAM**.
- La **GPU** (procesador gráfico): dibuja píxeles a gran velocidad. Trabaja con su propia memoria gráfica, la **VRAM** (*Video RAM*). [10]

En un PC de sobremesa, RAM y VRAM son chips físicamente distintos. En una Quest, el procesador es un SoC móvil con **memoria unificada**: CPU y GPU comparten los mismos chips de RAM. Aun así, la distinción *lógica* sigue siendo útil: una imagen "en la GPU" está en un formato y un lugar pensados para dibujar, y para que el código C# pueda leer sus bytes hay que **traerla a memoria de CPU** (eso es el *readback* de §4).

Las imágenes de las cámaras (`GetTexture()`) están **en la GPU**. Componerlas y guardarlas requiere moverlas por esta jerarquía con cuidado.

3.2 Los tres tipos de "textura" que verás

Una **textura** es, para la GPU, una imagen: una rejilla de píxeles con un formato de color (por ejemplo RGBA de 8 bits por canal = 4 bytes por píxel). En este proyecto conviven tres tipos y **no son intercambiables**:

Tipo	Qué es	Quién es su dueño
<code>Texture</code> (genérica)	La imagen prestada que entrega el SDK con <code>GetTexture()</code> .	El SDK de Meta. Puede sobrescribirla en cualquier momento; no debes conservarla , hay que copiar su contenido. Por eso <code>ParStereo</code> avisa: " <i>Vistas prestadas de la fuente</i> ".
<code>RenderTexture</code> (RT)	Un lienzo en la GPU sobre el que se puede dibujar.	Tu programa. Es donde se compone el SBS.
<code>Texture2D</code>	Una imagen que la CPU puede leer/escribir píxel a píxel (<code>ReadPixels</code> , <code>EncodeToPNG</code>).	Tu programa. Es el paso final antes de comprimir a archivo.

El **bloqueo B07** del plan describe justo el peligro de confundirlos: si guardas cien veces **la misma `RenderTexture`** en una lista y luego la sobrescribes, no tienes cien fotos distintas, tienes **cien referencias al mismo lienzo**, que muestran todas el último fotograma. (Este es el *mini-reto* del final del plan.) Para conservar una instantánea hay que **copiar** su contenido a un búfer propio, no guardar una referencia.

3.3 Shader y material

- Un **shader** es un pequeño programa que se ejecuta en la GPU y decide **el color de cada píxel** que se dibuja. [11]
- Un **material** es una instancia de un shader **con sus valores concretos** (qué texturas usa, qué parámetros). [12]

En este proyecto el shader compositor recibe dos texturas de entrada, llamadas por sus **propiedades** `_LeftTex` y `_RightTex`, y produce la imagen SBS colocando la izquierda en la mitad izquierda y la derecha en la mitad derecha. El **bloque B04** del plan era precisamente que el material escribía `_LeftEye` / `_RightEye` pero el shader declaraba `_LeftTex` / `_RightTex`: nombres que no coinciden ⇒ texturas que nunca llegan. En el código ya corregido, `CompositorStereo` usa `Shader.PropertyToID("_LeftTex")` y `...("_RightTex")`, que es la forma eficiente y sin errores de tipeo de nombrar esas propiedades.

3.4 Qué es un *blit*

"**Blit**" viene de *bit block transfer* (transferencia de un bloque de bits). Originalmente significaba "copiar un rectángulo de píxeles de un sitio a otro de la memoria gráfica de una sola vez". [13]

En Unity, `Graphics.Blit(origen, destino, material)` significa: "**dibuja una imagen que llena todo el destino, procesando cada píxel con el material (shader)**". Es la herramienta estándar para transformar o combinar imágenes en la GPU. [14]

En este proyecto el *blit* es el corazón de la composición estéreo. En `CompositorStereo`:

```
material.SetTexture(LeftTex, par.Izquierda);    // asigna la vista izquierda al shader
material.SetTexture(RightTex, par.Derecha);     // y la derecha
Graphics.Blit(null, rt, material);              // dibuja el SBS completo en la RenderTexture 'rt'
```

Con **una sola operación de GPU** se lee la vista izquierda, la derecha, y se pinta la imagen SBS combinada en `rt`. Es rapidísimo porque lo hace la GPU en paralelo para todos los píxeles.

También se usa un *blit* con escala para la **miniatura**: `Graphics.Blit(imagen, temporal, new Vector2(0.5f, 1), Vector2.zero)` toma solo la **mitad horizontal** (escala 0,5 en X) —es decir, un ojo— y la reduce, para no guardar la miniatura del par completo.

Por qué importa "esperar el final del frame". El SDK actualiza la textura de la cámara en el hilo de render. Si haces el *blit* en el instante equivocado, capturas el fotograma anterior. Por eso el código llama a `WaitForEndOfFrame` (envuelto en `EsperarFinalDeFrameAsync`) antes de componer: garantiza que la textura ya contiene el fotograma actual. [15]

3.5 Espacio de color: sRGB frente a lineal

Un matiz que aparece como `GL.sRGBWrite` y `RenderTextureReadWrite.sRGB`. Las imágenes se guardan en un espacio de color llamado **sRGB** (ajustado a cómo percibe el ojo), pero muchos cálculos de iluminación se hacen en espacio **lineal**. Si mezclas los dos sin convertir, los colores salen lavados o demasiado oscuros. El código ajusta `GL.sRGBWrite` según `QualitySettings.activeColorSpace` para que el *blit* escriba el color correcto. No necesitas dominarlo para entender el flujo; basta saber que **es un ajuste para que los colores se guarden fieles**. [16]

Referencias: VRAM [10]; shaders [11]; materiales [12]; *bit blit* [13]; `Graphics.Blit` [14]; `WaitForEndOfFrame` [15]; espacio de color lineal/gamma [16].

4. De la GPU a la CPU: *readback*, hilos y el porqué de B08/B09

4.1 El problema

Para **guardar** una imagen en un archivo PNG/JPEG o para **codificarla** en vídeo, el código (que corre en la CPU) necesita **los bytes** de los píxeles. Pero la imagen compuesta está en la **GPU** (una `RenderTexture`). Hay que **traerla a la CPU**. Esa operación se llama **GPU→CPU readback** ("relectura"). [17][18]

Hay dos formas:

Método	Cómo funciona	Coste
<code>Texture2D.ReadPixels</code>	Bloqueante: la CPU se para y espera a que la GPU termine y le entregue los píxeles.	Sencillo, pero congela el hilo ; sirve para una foto puntual, no para vídeo continuo. [18]
<code>AsyncGPUReadback</code>	Asíncrono: pides la lectura y sigues; cuando la GPU acaba, recoges los datos.	No congela; imprescindible para vídeo a varios FPS. [17]

En `CompositorStereo` verás que para foto se usa preferentemente `AsyncGPUReadback` (con `ReadPixels` como respaldo) y para vídeo se **exige** `AsyncGPUReadback` (`SystemInfo.supportsAsyncGPUReadback`), porque grabar exige leer muchos fotogramas sin trabar la app.

```
var solicitud = AsyncGPUReadback.Request(rt, 0, TextureFormat.RGBA32);
while (!solicitud.done) await Task.Yield(); // no bloquea: cede el turno hasta que la GPU acaba
imagen.LoadRawTextureData(solicitud.GetData<byte>()); // ya tenemos los bytes en la CPU
```

4.2 La regla de oro: las APIs gráficas van en el hilo principal

Un programa puede tener varios **hilos** (líneas de ejecución en paralelo). En Unity hay un **hilo principal** ("main thread") que es el único autorizado a tocar la mayoría de objetos gráficos (crear texturas, leer píxeles, etc.). Intentar hacerlo desde un hilo secundario (por ejemplo, dentro de `Task.Run`, que lanza

trabajo a otro hilo) provoca errores o comportamientos indefinidos. Unity ni siquiera permite crear texturas en segundo plano salvo que se active explícitamente. [19]

4.3 Qué eran exactamente B08 y B09

Son dos de los "bloqueos" catalogados en el plan (§3), y provienen del archivo antiguo

`FramePngProcessor.cs` :

- **B08** — "creación de `Texture2D`, acceso gráfico y destrucción dentro de `Task.Run`". Traducción: el código creaba texturas y leía píxeles **en un hilo secundario**, violando la regla del §4.2. Corrección prevista: separar la **lectura de GPU** (en el hilo principal) del **trabajo de CPU/archivos** (que sí puede ir en segundo plano). El código nuevo lo respeta: el *blit* y el *readback* se lanzan desde el hilo principal, y solo la escritura de bytes a disco se manda a `Task.Run`.
- **B09** — "destino 2×2, copia de un frame completo y posterior lectura CPU". Traducción: se creaba una textura destino de **2×2 píxeles** y se intentaba copiar en ella un fotograma completo y luego leer sus píxeles, sin un *readback* correcto. Resultado: **dimensiones incompatibles** (no caben los píxeles) y **datos basura**, porque faltaba la relectura GPU→CPU adecuada. Corrección: usar las **dimensiones reales** y un `AsyncGPUReadback / ReadPixels` explícito, que es justo lo que hace `CompositorStereo`.

El plan lo resume así: "La incompatibilidad de B08/B09 concuerda con las reglas de creación de texturas en el hilo principal y los requisitos de copia de Unity", y cita la documentación de creación de texturas y de copia de texturas de Unity. [19][20]

Nomenclatura: en el plan, **B** = *Bloqueo* (bug que impide compilar o rompe un objetivo) y el número es solo un identificador de lista (B01, B02, ...). Ver §12.

Referencias: `AsyncGPUReadback` [17]; `Texture2D.ReadPixels` [18]; creación de texturas en hilos [19]; `Graphics.CopyTexture` [20].

5. Emparejamiento estéreo: sincronizar el ojo izquierdo y el derecho

5.1 El problema del "emparejamiento"

Las dos cámaras son **independientes**: cada una entrega sus fotogramas con su propio reloj y a su propio ritmo. Si tomas la última imagen de la izquierda y la última de la derecha **sin comprobar cuándo se capturó cada una**, puedes acabar combinando un instante de la izquierda con **otro instante distinto** de la derecha. En una escena con movimiento, eso produce una profundidad falsa o imágenes "fantasma".

Emparejar (o "emparejamiento por tiempo") significa **elegir una vista izquierda y una derecha que correspondan al mismo instante**, dentro de una tolerancia. Es el equivalente software del *genlock* (sincronización de cámaras) del vídeo profesional. [21]

El plan lo pide en **F05**: "Evitar combinar imágenes de instantes distintos... Validar timestamps y actualización de ambos ojos".

5.2 Cómo se hace aquí: timestamps + tolerancia + monotonía

Cada textura de cámara viene con un **timestamp** (marca de tiempo, en *ticks* de `DateTime` ; 1 tick = 100 nanosegundos). La clase `ValidacionParStereo` decide si un par es válido con tres condiciones:

1. **Ambos son nuevos**: cada timestamp debe ser **mayor que el anterior** que ya usamos (`izquierda > anteriorIzquierda` y `derecha > anteriorDerecha`). Así nunca repites un fotograma ni retrocedes en el tiempo (monotonía).
2. **Son casi simultáneos**: la diferencia entre el timestamp izquierdo y el derecho debe caber dentro de una **tolerancia** (por defecto `toleranciaStereoMs = 20` ms). Si un ojo va muy retrasado respecto al otro, el par se rechaza.
3. **Son válidos** (mayores que cero, tolerancia no absurda).

```
long diferencia = izquierda ≥ derecha ? izquierda - derecha : derecha - izquierda;  
return diferencia ≤ toleranciaMilisegundos * TimeSpan.TicksPerMillisecond;
```

Un detalle fino y didáctico: el código **resta los ticks antes de convertirlos a decimal**, porque los ticks de `DateTime` son números enormes y, si los convirtieras a `double` por separado, perderías la precisión necesaria para medir diferencias de milisegundos. Es un ejemplo real de por qué el orden de las operaciones numéricas importa.

Si durante una grabación las cámaras dejan de entregar pares sincronizados durante demasiado tiempo, `GrabarVideoAsync` lanza un `TimeoutException` y **descarta el vídeo incompleto** en vez de guardar algo corrupto.

Referencias: sincronización de cámaras / *genlock* [21]; *timestamp* de presentación [22].

6. Memoria y tiempo: la sección clave

Esta es la parte del plan que planteaba más dudas. Vamos muy despacio.

6.1 Los tres lugares donde puede "llenarse" la memoria

Cuando el plan habla de memoria hay que distinguir **tres** cosas distintas:

Memoria	Qué guarda aquí	¿Se satura por grabar?
Memoria de vídeo (VRAM / memoria de GPU)	Las texturas vivas: las vistas de cámara y la <code>RenderTarget</code> del SBS.	Poco: se usan y liberan (<code>RenderTarget.GetTemporary / ReleaseTemporary</code>). Es un uso acotado.
Memoria del sistema (RAM)	Los búferes de bytes (<code>byte[] rgba</code>) de cada fotograma ya leído de la GPU, esperando a ser codificado.	Sí, es la que peligra. Si produces fotogramas más rápido de lo que el codificador los consume, se acumulan y la RAM se agota.
Almacenamiento (disco / flash)	El archivo final <code>.mp4</code> o <code>.png</code> ya escrito.	Se llena aparte, por tamaño del archivo. Es un límite distinto (prueba T10: "poco almacenamiento").

Respuesta directa a la duda del enunciado: cuando el plan dice "*sin acumular indefinidamente imágenes en memoria*" y "*al saturarse la cola*" se refiere a la **RAM del sistema**, no a la de vídeo ni al disco. La cola es una lista de **fotogramas ya convertidos a bytes** esperando su turno para codificarse; si crece sin límite, se agota la RAM y la app se cierra. El disco es un problema separado (que el archivo final no quepa), y la VRAM se mantiene acotada porque cada `RenderTarget` temporal se libera enseguida.

6.2 Cuánto ocupa el vídeo sin comprimir (por qué NO cabe en memoria)

Hagamos el cálculo del plan, que es puramente aritmético:

- Cada vista de un ojo: **1280 × 960** píxeles.
- Un par SBS: **2 × 1280 × 960 = 2 457 600** píxeles.
- Cada píxel en **RGBA** ocupa **4 bytes** (rojo, verde, azul, alfa; 1 byte cada uno).
- Un fotograma SBS sin comprimir: **2 × 1280 × 960 × 4 = 9 830 400 bytes ≈ 9,38 MiB.**

A **30 fotogramas por segundo (FPS)**, un minuto de vídeo **sin comprimir** serían:

- **9 830 400 bytes × 30 × 60 = 17 694 720 000 bytes ≈ 16,48 GiB.**

¡Más de 16 gigabytes por un solo minuto! Una Quest no tiene tanta memoria. De ahí la conclusión del plan: es **imposible** guardar todos los fotogramas en RAM y comprimir al final; hay que **codificar y liberar cada fotograma sobre la marcha**, con una **cola pequeña** y un presupuesto de memoria explícito.

MiB vs MB. El plan usa MiB/GiB (múltiplos de 1024: 1 MiB = 1 048 576 bytes), no MB/GB decimales (múltiplos de 1000). La diferencia importa al comparar cifras exactas. [23]

6.3 Qué es una "cola" y qué significa "saturarse"

Una **cola** (o *buffer*) es una lista de espera: el **productor** (el que genera fotogramas) los deja en la cola, y el **consumidor** (el codificador) los va sacando y comprimiendo. Es el clásico problema **productor–consumidor**. [24]

- Si el consumidor va **igual de rápido** que el productor, la cola se mantiene pequeña. Todo bien.
- Si el consumidor va **más lento** (codificar cuesta más que capturar), la cola **crece**. Cuando llega a su límite, se dice que está **saturada** o "llena".

Una cola **acotada** (con un tamaño máximo) obliga a decidir **qué hacer cuando se llena**. Esa presión del consumidor lento sobre el productor rápido se llama **contrapresión** (*backpressure*). [24]

Este proyecto elige la solución más estricta posible: una cola de capacidad 1. El comentario del codificador lo dice literalmente: "*Consumidor de capacidad uno: el productor espera cada llamada, sin acumular frames*". En `GrabarVideoAsync`, la línea:

```
await codificador.EscribirAsync(rgba, tiempoUs); // el productor NO sigue hasta que este frame
```

hace que el bucle de captura **espere** (`await`) a que cada fotograma se termine de meter en el codificador antes de capturar el siguiente. Así **la cola nunca crece**: hay como mucho un fotograma "en vuelo". Es *backpressure* llevado al extremo, y es la razón por la que la memoria se mantiene estable en la prueba T07/T08.

6.4 "Descartar al saturarse la cola": la política de fotogramas perdidos

Si el codificador no da abasto, el productor no puede esperar eternamente (congelaría la grabación). El plan enumera **tres políticas** válidas ante saturación (§4.2 del plan):

1. **Descartar** el fotograma que no cabe, **contándolo**, y **conservar la escala temporal** (que el vídeo dure lo que tiene que durar aunque le falten cuadros).
2. **Reducir la calidad** en una sesión posterior (menos resolución/FPS).
3. **Detener** con un error recuperable.

Este proyecto aplica la **política 1** de forma elegante: como espera a cada fotograma, si el hardware **no entregó** imágenes a tiempo, simplemente **no hubo fotograma** en ese intervalo, y el código lo registra como **omitido** en vez de inventarlo:

```
int omitidos = frames == 0 ? 0 : Math.Max(0, (int)Math.Floor((ahora - siguiente) / intervalo));
FramesOmitidos += omitidos; // se CUENTAN los que faltaron
siguiente += (omitidos + 1) * intervalo; // el reloj avanza como si hubieran pasado
```

El contador `FramesOmitidos` es lo que la interfaz muestra como "*N imágenes omitidas*". **Descartar = tirar ese fotograma pero seguir contando el tiempo correctamente**, para no deformar la duración del vídeo.

6.5 El tiempo: por qué NO hay que "fabricar FPS declarando 1/intervalo"

Aquí está la frase que más dudas generaba. Vamos parte por parte.

- **FPS** = *frames per second*, fotogramas por segundo. Si quieres 30 FPS, quieres un fotograma cada `1/30` `≈ 0,0333` segundos. Ese "cada cuánto" es el **intervalo**: `intervalo = 1 / FPS`. Al revés, `FPS = 1 / intervalo`. En el código: `double intervalo = 1.0 / perfil.Fps;`

- **"Fabricar FPS declarando 1/intervalo"** sería la **trampa** que el plan prohíbe: consiste en **asumir** que cada fotograma que guardas está separado exactamente **1/FPS** del anterior y ponerle al fotograma número n la marca de tiempo **$n \times (1/\text{FPS})$** , **aunque el hardware no haya producido esos fotogramas a ese ritmo**. Es "fabricar" (inventar) un ritmo constante que en realidad no ocurrió.

El problema: si el visor se calienta o va lento y solo entrega 18 FPS reales, pero tú marcas los cuadros como si fueran 30 FPS perfectos, el vídeo resultante irá **acelerado** y su duración no coincidirá con el tiempo real de grabación. El movimiento mentiría.

- **Lo correcto** (lo que hace el código) es usar **timestamps de tiempo real** medidos con un reloj fiable:

```
csharp double ahora = Time.realtimeSinceStartupAsDouble; // reloj real, no afectado por
cámara lenta long tiempoUs = (long)((ahora - inicio) * 1000000); // marca de tiempo REAL de
este frame, en microsegundos await codificador.EscribirAsync(rgba, tiempoUs);
```

Cada fotograma se sella con **el instante real en que se capturó** (**tiempoUs**, en microsegundos = PTS, *Presentation Time Stamp* [22]). El codificador MP4 respeta esos tiempos, así que la duración y la velocidad del vídeo coinciden con la realidad, **aunque falten fotogramas**. El plan lo dice tal cual: *"No fabricar FPS declarando **1 / intervalo** si el hardware no produjo esos frames. El reloj de la grabación y sus timestamps deben seguir representando el tiempo real aunque baje el rendimiento."*

- **Reloj monotónico.** Se usa **Time.realtimeSinceStartupAsDouble** (tiempo real desde el arranque) y **no Time.time**, porque **Time.time** puede escalarse o pausarse con el juego. Para medir tiempo de grabación necesitas un reloj que **solo avance y siempre al mismo ritmo** (monotónico). [25]
- **Timelapse vs. tiempo real.** El plan distingue el vídeo normal (tiempo real) de un posible *timelapse* futuro (donde sí quieres acelerar el tiempo a propósito). Son cosas distintas y no hay que confundir un bajón de rendimiento con un timelapse. [26]

6.6 Resumen visual: cómo se graba y se almacena un vídeo, paso a paso

Por cada fotograma, mientras grabas:

- [1] Esperar fin de frame (WaitForEndOfFrame) — asegura textura actual (§3.4)
- [2] Pedir par L/R al proveedor — ProveedorCamarasMeta.IntentarObtener()
- [3] Emparejar por tiempo — ValidacionParStereo (§5); si no valida, se salta
- [4] Contar fotogramas omitidos y avanzar el reloj — FramesOmitidos (§6.4)
- [5] BLIT: componer SBS en una RenderTexture (GPU) — CompositorStereo (§3.4)
- [6] READBACK: traer los bytes RGBA a la CPU — AsyncGPUReadback (§4)
- [7] Sellar con timestamp REAL en microsegundos — tiempoUs (§6.5)
- [8] ESCRIBIR al codificador y ESPERAR — CodificadorVideoAndroid.EscribirAsync (cola=1, §6.3)
 (dentro, en Android:)
 RGBA → YUV420 → MediaCodec (comprime a H.264/HEVC) → MediaMuxer (mete en MP4)
- [9] Repetir hasta "Detener"

Al detener:

- [10] finish(): cerrar el flujo, fijar la duración exacta del último fotograma
- [11] El MP4 estaba en una carpeta TEMPORAL oculta
- [12] Validar que el archivo existe y no está vacío — SesionVideo.PublicarAsync (§9)
- [13] Directory.Move: PUBLICAR (renombrar la carpeta) → ahora sí aparece en la galería

Nada se guarda "para el final": cada fotograma se **comprime y se suelta** enseguida (paso 8), y solo el archivo comprimido va creciendo en **disco**. La RAM se mantiene plana.

Referencias: VRAM [10]; prefijos binarios MiB/GiB [23]; productor–consumidor / *backpressure* [24]; reloj/tiempo en Unity [25]; *time-lapse* [26]; *timestamp* de presentación (PTS) [22].

7. Codificación de vídeo: códec, contenedor, MediaCodec y YUV

Grabar vídeo de verdad exige **comprimir**, porque (§6.2) sin comprimir no cabe. Aquí van los conceptos.

7.1 Códec ≠ contenedor (una confusión muy habitual)

- Un **códec** es el **algoritmo de compresión** de las imágenes. Los dos de este proyecto son **H.264** (también llamado **AVC**, MIME `video/avc`) y **HEVC** (**H.265**, MIME `video/hevc`). HEVC comprime mejor (archivos más pequeños a igual calidad) pero no todos los dispositivos lo admiten. [27][28]
- Un **contenedor** (o formato de archivo) es la "caja" que **envuelve** el vídeo comprimido, junto con metadatos, pistas de audio, tiempos, etc. Aquí es **MP4**. [29]

- La **disposición estereoscópica** (SBS) es **independiente** de las dos anteriores: describe cómo están colocadas las dos vistas dentro de cada fotograma.

El plan insiste en no mezclarlos: un perfil se describe como "MP4 · H.264 · 3D SBS" = contenedor · códec · disposición. Y avisa: guardar un archivo `.mov` o poner un nombre no demuestra compatibilidad real; hay que **probar** que un reproductor lo abre.

7.2 MediaCodec y MediaMuxer (las dos piezas de Android)

Android ofrece dos herramientas de bajo nivel que este proyecto usa desde Java (`TFGVideo.java`), llamadas desde C# por JNI (§10.4):

- **MediaCodec**: da acceso a los **codificadores por hardware** del chip. Le entregas fotogramas crudos (YUV) y te devuelve datos **comprimidos** (H.264/HEVC). Usar el hardware es lo que hace viable grabar en tiempo real sin fundir la batería. [30]
- **MediaMuxer**: **empaqueta** (multiplexa, de ahí "muxer") esos datos comprimidos dentro del archivo **MP4**, en una "pista" (*track*). [31]

Flujo real dentro de `TFGVideo.frame( ... )`:

1. Se pide un búfer de entrada al códec (`dequeueInputBuffer`).
2. Se **convierte el RGBA a YUV420** (§7.4) y se rellena ese búfer.
3. `queueInputBuffer( ... , ptsUs, ... )`: se entrega con su **timestamp** (§6.5).
4. `drain( ... )`: se sacan los datos ya comprimidos y `muxer.writeSampleData( ... )` los escribe en el MP4.
5. Al terminar, `finish()` envía una marca de **fin de flujo** (`BUFFER_FLAG_END_OF_STREAM`) y una muestra vacía que **fija la duración** del último fotograma; luego `muxer.stop()`.

7.3 Parámetros del formato de vídeo (qué significan)

En `MediaFormat` se configuran, entre otros:

Parámetro	Qué controla
<code>KEY_BIT_RATE</code> (bitrate)	Cuántos bits por segundo se dedican al vídeo. Más bitrate = más calidad y más tamaño. Aquí se calcula acotado entre 1 y 6 Mbit/s. [32]
<code>KEY_FRAME_RATE</code> (FPS)	Fotogramas por segundo objetivo.
<code>KEY_I_FRAME_INTERVAL</code>	Cada cuánto se inserta un fotograma clave (<i>I-frame</i>): una imagen completa que no depende de otras. Entre ellos van fotogramas que solo guardan diferencias . Un valor de 1 = un keyframe por segundo, lo que facilita buscar/reproducir. Al conjunto entre dos keyframes se le llama GOP (<i>Group of Pictures</i>). [33][34]
<code>KEY_COLOR_FORMAT = YUV420Flexible</code>	El codificador quiere el color en YUV , no en RGBA (§7.4).
<code>KEY_COLOR_STANDARD/RANGE/TRANSFER</code>	Detalles de cómo interpretar el color (BT.601, rango limitado, SDR) para que el vídeo se vea con los colores correctos.

7.4 Por qué se convierte RGBA → YUV (y qué es el submuestreo de croma)

Las pantallas y las texturas usan **RGB** (rojo, verde, azul). Pero los códecs de vídeo trabajan en **YUV** (o YCbCr): separan el **brillo** (Y, *luma*) del **color** (U y V, *croma*). [35]

¿Por qué? Porque el ojo humano distingue mucho mejor los cambios de **brillo** que los de **color**. Así, el vídeo guarda el brillo a resolución completa pero el color a **la mitad** en cada eje (formato **4:2:0**): eso es el **submuestreo de croma**, y ahorra memoria sin que se note apenas. [36]

En `RgbaYuv.java` verás justo eso: el plano **Y** se calcula para **cada** píxel, pero **U** y **V** solo para **uno de cada bloque 2×2** (promediando los cuatro), que es la definición de 4:2:0. Además, el bucle recorre las filas **de abajo arriba** (`height - 1 - y`) porque **Unity numera las texturas desde abajo y el vídeo desde arriba**: si no se invirtiera, el vídeo saldría del revés. Y respeta los **strides** de cada plano (`rowStride`, `pixelStride`) porque en memoria las filas pueden tener relleno.

Frame-packing. El plan menciona la opción *frame-packing* de x265: es una etiqueta que declara que un vídeo lleva dos vistas empaquetadas (por ejemplo SBS) para que el reproductor lo sepa. Es **metadato de disposición**, no un códec distinto. El proyecto define su base como SBS y comprueba la compatibilidad aparte. [37]

Referencias: H.264/AVC [27]; HEVC/H.265 [28]; MP4 [29]; `MediaCodec` [30]; `MediaMuxer` [31]; bitrate [32]; I-frame/keyframe [33]; GOP [34]; YCbCr [35]; submuestreo de croma [36]; frame-packing (x265) [37].

8. Foto: PNG frente a JPEG y los metadatos

8.1 PNG vs JPEG

- **PNG:** compresión **sin pérdida** (*lossless*): cada píxel se reconstruye exacto. Ideal para calidad máxima; archivos más grandes. [38]
- **JPEG:** compresión **con pérdida** (*lossy*): descarta detalle imperceptible para achicar el archivo; tiene un parámetro de **calidad** (1–100). El código expone `CalidadJpeg` (por defecto 90). [39]

En el código, tras el *readback*, `imagen.EncodeToPNG()` o `imagen.EncodeToJPG(calidad)` produce los bytes del archivo. El objetivo **O2** ("elección de formato") se cumple ofreciendo al menos PNG o JPEG y recordando la preferencia (`PlayerPrefs`).

8.2 Metadatos estéreo: iTXt, CRC y por qué fallaban F03/F04

Un PNG puede llevar **metadatos de texto** en trozos llamados **chunks**. El chunk `iTXt` guarda texto internacional (UTF-8) y **tiene una estructura obligatoria** de campos (palabra clave, banderas de compresión, idioma, etc.). [40]

- **F03:** el prototipo construía el `iTXt` **solo con la palabra clave y un XML**, sin los demás campos obligatorios ⇒ un lector estricto lo rechaza. Corrección: escribir la estructura completa y **probarla con**

un lector real.

- **CRC**: cada chunk lleva un **CRC** (código detector de errores). Que el CRC sea correcto solo garantiza que los bytes no se corrompieron; **no** garantiza que el *contenido* sea válido. El plan lo subraya: "el CRC correcto no hace válido el contenido".
- **F04 / XMP**: los metadatos de "esto es una foto 3D SBS" suelen ir en **XMP** (un estándar de metadatos incrustados). Poner campos con nombres bonitos pero vacíos **no** demuestra que ningún visor reconozca la foto como estéreo; hay que definir bien el SBS izquierda-derecha y **probar la profundidad**. [41]

Referencias: PNG [38]; JPEG [39]; especificación PNG, chunk iTXt (W3C) [40]; XMP [41].

9. Almacenamiento y publicación

9.1 Dónde guarda la app sus archivos

Unity da a cada app una carpeta privada y persistente: `Application.persistentDataPath`. Ahí se crea `stereo_captures`, el "almacén" propio de la aplicación. [42]

9.2 Escritura atómica: primero temporal, luego publicar

Un principio importante del plan: **no anunciar "Guardado" hasta que el archivo esté completo y verificado**, y no dejar archivos a medias si algo falla. La clase `SesionVideo` lo implementa así:

1. Escribe el MP4 en una carpeta **temporal oculta** (nombre que empieza por `.`).
2. Al terminar, **valida** que el archivo existe y no está vacío (`Length < 32` ⇒ error).
3. Solo entonces `Directory.Move` renombra la carpeta a su nombre definitivo. Renombrar es una operación **atómica**: o está el archivo entero o no está; nunca a medias.

Si la grabación se cancela, la carpeta temporal se borra (`Dispose`) y no queda basura. Esto responde a F07 ("directorios únicos por sesión") y F09 ("no anunciar Guardado antes de terminar").

9.3 Exportar a la galería del sistema: MediaStore, URI e `is_pending`

Una cosa es guardar en la **carpeta privada** de la app y otra **publicar** la captura en la **galería** de Android para que otras apps la vean. Desde Android moderno, las apps ya **no escriben libremente** en el almacenamiento común: usan **MediaStore**, pidiendo al sistema que cree una entrada y les dé una **URI** (una dirección) donde volcar los bytes. Esto es el **almacenamiento por ámbitos** (*scoped storage*). [43][44]

`ExportadorCapturas` hace el patrón canónico:

1. `ContentResolver.insert( ... )` en `MediaStore` con `is_pending = 1` (marca "aún no está lista, no la muestres").
2. Abre la URI, **copia los bytes** del archivo local.
3. Pone `is_pending = 0` (ya es visible). Si algo falla, **borra** la entrada (`delete`) para no dejar un archivo corrupto en la galería.

Se guarda en `Movies/TFG` (vídeo) o `Pictures/TFG` (foto) con `relative_path`. El plan recalca que exportar es **una operación aparte**: una exportación fallida **no** debe borrar la copia local correcta ni cantar victoria antes de tiempo (F09, punto 2 de §5.4).

9.4 Permisos

Android pide permisos **en tiempo de ejecución** (el usuario los concede/deniega mientras usa la app), no solo al instalar. Este proyecto separa dos permisos distintos: el de **cámara** (`HEADSET_CAMERA`, §2) y el de **almacenamiento**, y contempla que el usuario pueda **consultar la galería aunque no haya dado permiso de cámara**. [7]

Referencias: `Application.persistentDataPath` [42]; `MediaStore` [43]; `scoped storage` [44]; permisos en tiempo de ejecución [7].

10. Arquitectura, estados y asincronía

10.1 El "director de orquesta": `GestorCaptura` y `MonoBehaviour`

El plan usa la analogía del **director de orquesta**: `GestorCaptura` no lo hace todo, pero **da las órdenes y controla el estado**; el proveedor de cámaras, el compositor, el codificador y el almacén son los "músicos".

`GestorCaptura` es un `MonoBehaviour`: la clase base de Unity para componentes que se enganchan a objetos de la escena y reciben eventos del ciclo de vida (`Awake`, `Update`, `OnEnable`, `OnDestroy`, `OnApplicationPause` ...). [45] Los campos marcados con `[SerializeField]` son los que aparecen en el Inspector de Unity para asignarlos a mano (las dos cámaras, el material, los tiempos de espera). [46]

Bloqueos B01–B04 y los ficheros `.meta`. El plan detecta que la clase se llamaba `StereoFrameCapture` dentro de un archivo `GestorCaptura.cs` (inconsistencia grave en Unity). Además, cada asset en Unity tiene un archivo `.meta` con un **GUID** (identificador único); las escenas referencian los scripts por ese GUID, no por el nombre. Por eso el plan es tan cuidadoso al renombrar/borrar: hay que **conservar los `.meta`** o las referencias de la escena se rompen. [47]

10.2 Máquina de estados: una sola puerta para los comandos

La app está siempre en **uno** de estos estados (`EstadoCaptura`): `Inicializando`, `SinPermiso`, `Listo`, `CapturandoFoto`, `Guardando`, `Grabando`, `Finalizando`, `Suspendido`, `Error`. Una **máquina de estados** es un modelo donde solo se permiten **ciertas transiciones**. [48]

`ControlCaptura` es la "puerta única": por ejemplo, `IntentarCapturar()` solo deja empezar una foto **si el estado es `Listo`**. Esto evita, por ejemplo, que un doble clic o una llamada desde otro sitio inicien **dos grabaciones a la vez**. El plan lo pide expresamente ("validar la transición, además de deshabilitar el botón").

10.3 Asincronía: `async` / `await`, `Task` y cancelación

Capturar, leer de la GPU, codificar y escribir a disco **llevan tiempo**. Si se hicieran de golpe en el hilo principal, la app se **congelaría**. La solución es la **programación asíncrona**: `async` / `await` permite "esperar" a que algo termine **sin bloquear**, cediendo el turno mientras tanto. [49]

- Un `Task` representa "un trabajo que terminará en el futuro". [50]
- Un `CancellationToken` es un "botón de cancelar" que se pasa a esas tareas; si el usuario descarta el vídeo o suspende la app, se llama a `operacion.Cancel()` y las tareas lo detectan con `ThrowIfCancellationRequested()`. [51]
- `async void`: el código lo usa **solo** en `IniciarOperacion`, el adaptador que conecta con los `UnityEvent` de los botones, y **captura todas las excepciones** ahí. La regla general (que el plan enuncia) es "reservar `async void` para adaptadores de eventos que capturen sus excepciones", porque un `async void` que revienta sin capturar puede tumbar la app. [49]

10.4 Hilos y JNI: hablar con Java desde C

El codificador de Android está escrito en **Java** (`TFGVideo.java`). Unity llama a Java mediante **JNI** (*Java Native Interface*) a través de `AndroidJavaObject` / `AndroidJavaClass`. [52][53] Como esas llamadas se lanzan en un hilo de segundo plano (`Task.Run`), hay que **registrar ese hilo en la máquina virtual de Java** con `AndroidJNI.AttachCurrentThread()` y soltarlo al acabar (`DetachCurrentThread()`). Es un requisito técnico de JNI, no un capricho.

10.5 Ciclo de vida: suspender y reanudar

Si el usuario se quita el visor o cambia de app, Android llama a `OnApplicationPause(true)`. El código entonces **cancela** la operación en curso, **descarta** un vídeo a medio grabar (no se puede garantizar su integridad) y pasa a `Suspendido`; al volver, **revalida** cámaras y permisos. Esto cubre la prueba T09.

Referencias: `MonoBehaviour` [45]; `SerializeField` [46]; ficheros `.meta` de Unity [47]; máquina de estados finitos [48]; `async/await` en C# [49]; `Task` [50]; `CancellationToken` [51]; JNI de Android [52]; `AndroidJavaObject` de Unity [53].

11. La interfaz en realidad virtual

- **uGUI** es el sistema de interfaz clásico de Unity (botones, paneles, texto); **TextMeshPro** es el componente de texto de alta calidad. [54][55]
- Un **Canvas en espacio mundial** (*world-space*) es un panel de interfaz que **flota en el espacio 3D** delante del usuario, en lugar de pegado a la pantalla (que en VR no existe). [54]
- Para pulsar botones en VR hace falta un `EventSystem` más un **raycaster**: se lanza un "rayo" desde el mando y, donde toca, se activa el botón. El plan advierte que tener `EventSystem` **no demuestra** que se pueda pulsar en el dispositivo; hay que probarlo con los mandos. [56]
- El **visor estéreo** reutiliza la idea de §1: para cada ojo, el shader muestrea la mitad de la textura SBS que le toca (`[0, 0.5]` al izquierdo, `[0.5, 1]` al derecho). El plan exige validarlo con una imagen de

prueba que ponga "L" solo al ojo izquierdo y "R" solo al derecho —justo los patrones que genera `ProveedorCamarasMeta.CrearPatron` en el Editor— y luego con una escena real para comprobar profundidad.

Referencias: uGUI / Canvas [54]; TextMeshPro [55]; EventSystem [56].

12. Cómo leer el plan

Para que el vocabulario del `PLAN_IMPLEMENTACION.md` no despiste:

Marca	Significado
O1–O4	Objetivos del TFG (foto/vídeo estéreo, elección de formato, galería/visor, grabación continua). Cada uno tiene un criterio de aceptación .
B01–B12	Bloqueos (<i>bugs</i> que impiden compilar o rompen un objetivo). El número es solo un índice de lista. Ej.: B08/B09 (\$4.3).
F01–F14	Fallos de integridad o funciones ausentes (menos graves que un bloqueo, pero necesarios). Ej.: F05 (emparejamiento), F08 (tiempo/FPS).
T01–T15	Pruebas de aceptación: escenarios concretos con resultado esperado que demuestran que un objetivo se cumple.
P0 / P1 / P2	Prioridad: P0 bloquea compilar/arrancar; P1 impide un objetivo principal; P2 afecta a robustez o mantenimiento.
"Confirmado"	Visible en el código o la serialización, no necesariamente reproducido en hardware. Es honestidad sobre el alcance del análisis.
Fase 0–6	Orden de implementación por dependencias: inventario → captura física → foto/almacenamiento → vídeo → interfaz → galería/visor → validación. Cada fase tiene una "puerta de salida" (sus pruebas).

La lógica de fondo del plan es **construir de abajo arriba**: no tiene sentido diseñar pantallas bonitas sobre una captura que aún no produce archivos válidos. Primero que funcione la base (par estéreo real), luego foto y almacenamiento, luego vídeo, y solo al final la interfaz y el visor.

13. Glosario rápido

Término	En una frase
Estereoscopia / estéreo	Dos vistas ligeramente distintas que, una por ojo, crean sensación de profundidad. (§1)
SBS (<i>side-by-side</i>)	Guardar las dos vistas una al lado de la otra en la misma imagen. (§1)
Passthrough	Ver el exterior real dentro del visor mediante sus cámaras. (§2)
PCA (Passthrough Camera API)	La API que deja leer las imágenes de esas cámaras. (§2)
Textura	Una imagen tal como la maneja la GPU. (§3)
RenderTarget	Un lienzo de GPU sobre el que se dibuja (aquí, el SBS). (§3)
Blit	Dibujar/combinar imágenes en la GPU en una pasada, a través de un shader. (§3.4)
Shader / material	Programa que colorea píxeles / instancia con valores concretos. (§3.3)
Readback	Traer los píxeles de la GPU a la memoria de CPU para guardarlos. (§4)
Hilo principal	El único hilo autorizado a tocar objetos gráficos en Unity. (§4.2)
Emparejamiento	Elegir vista izquierda y derecha del mismo instante (por timestamp y tolerancia). (§5)
Timestamp / PTS	Marca del instante de un fotograma; el vídeo la usa para respetar el tiempo real. (§5, §6.5)
VRAM / RAM / disco	Memoria de GPU / memoria de sistema / almacenamiento. La cola se satura en la RAM . (§6.1)
Cola / buffer	Lista de espera de fotogramas entre el que los produce y el que los codifica. (§6.3)
Saturarse / backpressure	La cola se llena porque el codificador va más lento; hay que descartar o esperar. (§6.3–6.4)
Fabricar FPS 1/intervalo	Trampa prohibida: fingir un ritmo constante ignorando el tiempo real. (§6.5)
Códec (H.264/HEVC)	Algoritmo que comprime el vídeo. (§7.1)
Contenedor (MP4)	La "caja" de archivo que envuelve el vídeo comprimido. (§7.1)
MediaCodec / MediaMuxer	Codificador por hardware de Android / empaquetador en MP4. (§7.2)
YUV / 4:2:0	Separar brillo y color; guardar el color a media resolución. (§7.4)
Bitrate	Bits por segundo de vídeo: más = mejor calidad y más tamaño. (§7.3)
I-frame / GOP	Fotograma completo de referencia / grupo entre dos de ellos. (§7.3)
PNG / JPEG	Foto sin pérdida / con pérdida (y calidad ajustable). (§8.1)

Término	En una frase
iTt / CRC / XMP	Metadatos de texto en PNG / detector de errores / metadatos estándar incrustados. (§8.2)
MediaStore / URI / scoped storage	Cómo Android publica y localiza archivos en la galería común. (§9.3)
MonoBehaviour / .meta / GUID	Componente de Unity / archivo que identifica un asset / su identificador único. (§10.1)
Máquina de estados	Modelo con estados y transiciones permitidas que evita órdenes inválidas. (§10.2)
async/await / Task / CancellationToken	Esperar sin congelar / trabajo futuro / botón de cancelar. (§10.3)
JNI	Puente para llamar a código Java desde C#. (§10.4)

14. Referencias consolidadas

Cada enlace respalda el concepto donde aparece citado con su número.

1. Estereoscopia — <https://es.wikipedia.org/wiki/Estereoscop%C3%ADa>
2. Disparidad binocular — https://en.wikipedia.org/wiki/Binocular_disparity
3. Distancia interpupilar — https://en.wikipedia.org/wiki/Pupillary_distance
4. Passthrough Camera API (Meta, Unity) — <https://developers.meta.com/horizon/documentation/unity/unity-pca-documentation/>
5. Mixed Reality Utility Kit (MRUK) — <https://developers.meta.com/horizon/documentation/unity/unity-mr-utility-kit-overview/>
6. Passthrough (Meta) — <https://developers.meta.com/horizon/documentation/unity/unity-passthrough/>
7. Permisos en tiempo de ejecución (Android) — <https://developer.android.com/guide/topics/permissions/overview>
8. Universal Render Pipeline (Unity) — <https://docs.unity3d.com/Manual/urp/urp-introduction.html>
9. OpenXR (Khronos) — <https://www.khronos.org/openxr/>
10. Video RAM (VRAM) — https://en.wikipedia.org/wiki/Video_random-access_memory
11. Shaders (Unity, manual) — <https://docs.unity3d.com/Manual/Shader.html>
12. Materiales (Unity, manual) — <https://docs.unity3d.com/Manual/Materials.html>
13. Bit blit — https://en.wikipedia.org/wiki/Bit_blit
14. `Graphics.Blit` (Unity) — <https://docs.unity3d.com/ScriptReference/Graphics.Blit.html>
15. `WaitForEndOfFrame` (Unity) — <https://docs.unity3d.com/ScriptReference/WaitForEndOfFrame.html>
16. Trabajo en color lineal o gamma (Unity) — <https://docs.unity3d.com/Manual/LinearRendering-LinearOrGammaWorkflow.html>
17. `AsyncGPUReadback` (Unity) — <https://docs.unity3d.com/ScriptReference/Rendering.AsyncGPUReadback.html>
18. `Texture2D.ReadPixels` (Unity) — <https://docs.unity3d.com/ScriptReference/Texture2D.ReadPixels.html>

19. `Texture.allowThreadedTextureCreation` (Unity) — <https://docs.unity3d.com/ScriptReference/Texture-allowThreadedTextureCreation.html>
20. `Graphics.CopyTexture` (Unity) — <https://docs.unity3d.com/ScriptReference/Graphics.CopyTexture.html>
21. Genlock (sincronización de vídeo) — <https://en.wikipedia.org/wiki/Genlock>
22. Presentation timestamp (PTS) — https://en.wikipedia.org/wiki/Presentation_timestamp
23. Prefijos binarios (MiB, GiB) — https://en.wikipedia.org/wiki/Binary_prefix
24. Problema productor–consumidor — https://en.wikipedia.org/wiki/Producer%E2%80%93consumer_problem
25. `Time` (Unity, tiempo y reloj) — <https://docs.unity3d.com/ScriptReference/Time-realtimeSinceStartupAsDouble.html>
26. Time-lapse — https://en.wikipedia.org/wiki/Time-lapse_photography
27. H.264 / AVC — https://en.wikipedia.org/wiki/Advanced_Video_Coding
28. HEVC / H.265 — https://en.wikipedia.org/wiki/High_Efficiency_Video_Coding
29. MP4 — https://en.wikipedia.org/wiki/MP4_file_format
30. `MediaCodec` (Android) — <https://developer.android.com/reference/android/media/MediaCodec>
31. `MediaMuxer` (Android) — <https://developer.android.com/reference/android/media/MediaMuxer>
32. Tasa de bits (bitrate) — https://en.wikipedia.org/wiki/Bit_rate
33. Tipos de fotograma (I/P/B, keyframe) — https://en.wikipedia.org/wiki/Video_compression_picture_types
34. Group of pictures (GOP) — https://en.wikipedia.org/wiki/Group_of_pictures
35. YCbCr / YUV — <https://en.wikipedia.org/wiki/YCbCr>
36. Submuestreo de croma (4:2:0) — https://en.wikipedia.org/wiki/Chroma_subsampling
37. Frame packing (x265) — <https://x265.readthedocs.io/en/master/cli.html#cmdoption-frame-packing>
38. PNG — <https://en.wikipedia.org/wiki/PNG>
39. JPEG — <https://en.wikipedia.org/wiki/JPEG>
40. Especificación PNG, chunk `iTXt` (W3C) — <https://www.w3.org/TR/png-3/#11iTXt>
41. Extensible Metadata Platform (XMP) — https://en.wikipedia.org/wiki/Extensible_Metadata_Platform
42. `Application.persistentDataPath` (Unity) — <https://docs.unity3d.com/ScriptReference/Application-persistentDataPath.html>
43. `MediaStore` (Android) — <https://developer.android.com/reference/android/provider/MediaStore>
44. Almacenamiento por ámbitos (scoped storage) — <https://developer.android.com/training/data-storage#scoped-storage>
45. `MonoBehaviour` (Unity) — <https://docs.unity3d.com/ScriptReference/MonoBehaviour.html>
46. `SerializeField` (Unity) — <https://docs.unity3d.com/ScriptReference/SerializeField.html>
47. Ficheros `.meta` y metadatos de asset (Unity) — <https://docs.unity3d.com/Manual/AssetMetadata.html>
48. Máquina de estados finitos — https://en.wikipedia.org/wiki/Finite-state_machine
49. Programación asíncrona con `async/await` (C#, Microsoft) — <https://learn.microsoft.com/dotnet/csharp/asynchronous-programming/>
50. `Task` (.NET, Microsoft) — <https://learn.microsoft.com/dotnet/api/system.threading.tasks.task>
51. `CancellationToken` (.NET, Microsoft) — <https://learn.microsoft.com/dotnet/api/system.threading.cancellationtoken>

- 52. JNI — <https://developer.android.com/training/articles/perf-jni>
 - 53. `AndroidJavaObject` (Unity) — <https://docs.unity3d.com/ScriptReference/AndroidJavaObject.html>
 - 54. Canvas / uGUI (Unity) — <https://docs.unity3d.com/Manual/UICanvas.html>
 - 55. TextMeshPro (Unity) — <https://docs.unity3d.com/Manual/com.unity.ugui.html>
 - 56. Event System (Unity) — <https://docs.unity3d.com/Manual/EventSystem.html>
-

Guía redactada como material de estudio para el TFG de captura estereoscópica en Meta Quest. Se basa en el `PLAN_IMPLEMENTACION.md` y en el código de `Assets/Scripts` y `Assets/AndroidPlugins`. Los conceptos se explican a nivel introductorio; para dominarlos, sigue los enlaces de referencia y contrástalos con la versión del SDK instalada en el proyecto.